AI Usage & Implementation Notes
Project: FlakyTest Detector Engine
Team: Team Syndicate

Overview
This document summarizes how Artificial Intelligence tools were utilized during the design, development, testing, and refinement of the FlakyTest Detector Engine. AI was used as a collaborative engineering assistant to accelerate development, improve code quality, and support architectural decision-making. All generated outputs were reviewed, validated, and integrated by the development team.

1. AI Tools & Models Utilized
• Claude & Gemini – Architecture design, frontend development assistance, and backend debugging support.
• ChatGPT – Unit test generation, parser optimization, documentation support, and implementation guidance.
• Ollama (Llama 3 8B) – Primary local AI agent used for failure-log analysis and root-cause hypothesis generation.
• Groq Cloud (Llama 3.1) – High-speed cloud fallback model providing continuity when local inference is unavailable.

2. Key Areas of AI Assistance

Architecture & Backend Development (FastAPI)
AI assisted in structuring the FastAPI backend, designing the orchestration workflow, connecting ingestion pipelines with the flakiness analysis engine, and implementing local/cloud LLM failover logic.

Frontend UI/UX (React + Vite)
AI accelerated frontend development by generating component boilerplates, responsive layouts, dashboard structures, and the “Chat with Data” interface integrated with backend APIs.

Universal Data Parser Engine
AI contributed to the design of a schema-agnostic parser capable of processing CSV, JSON, and Excel files while automatically inferring column mappings through data heuristics.

Testing & Quality Assurance
AI generated an extensive pytest suite covering parser validation, flaky-test detection, error handling, regression scenarios, and edge cases. The generated tests were refined and validated by the team to achieve reliable coverage.

Prompt Engineering for Root-Cause Analysis
AI-assisted prompt iterations helped establish a structured output format requiring a Likely Root Cause, Confidence Percentage, and Recommended Fix for every analysis result.

3. Challenges & Human Interventions

Frontend Integration Mismatches
AI occasionally suggested incorrect API routes and frontend integrations. These discrepancies were manually identified and corrected by the team.

LLM Output Formatting Issues
Early model outputs were inconsistent and overly verbose. Manual prompt refinement and backend

validation logic were introduced to enforce structured responses.

Library & Framework Hallucinations
AI occasionally recommended deprecated libraries or incorrect framework methods. Official documentation and manual testing were used to verify and implement correct solutions.

4. Human Oversight & Validation
All AI-generated code, architecture suggestions, prompts, tests, and documentation underwent manual review. The development team was responsible for implementation decisions, debugging, integration, validation, and final quality assurance.

Conclusion
For Team Syndicate, AI functioned as a highly effective engineering assistant and productivity accelerator. While AI significantly reduced development time for boilerplate generation, testing, documentation, and prototyping, successful delivery of the FlakyTest Detector Engine required substantial human expertise, system integration, debugging, validation, and decision-making.